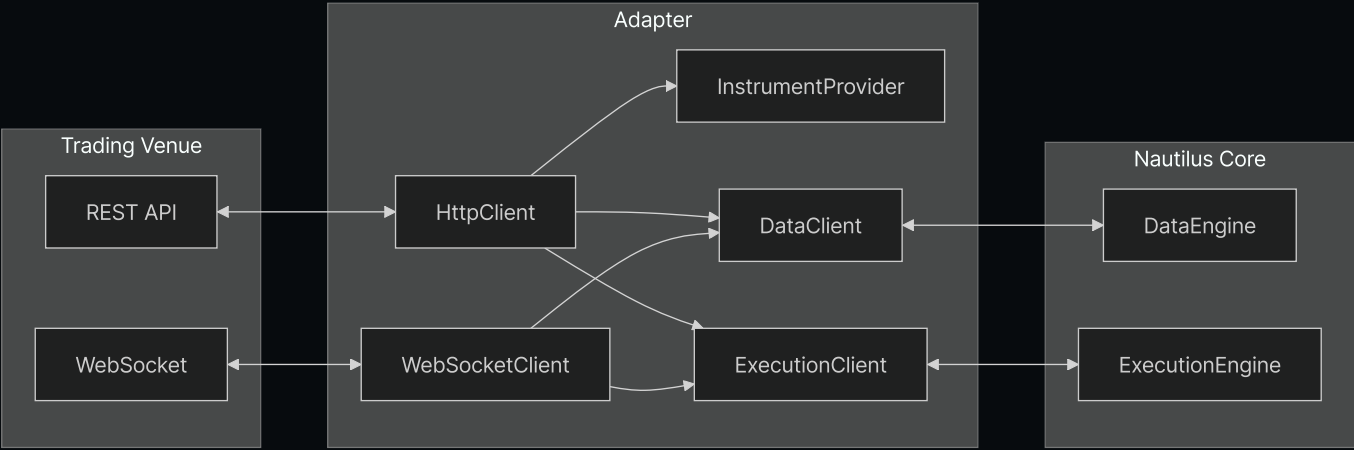


Concepts

Adapters

Adapters integrate data providers and trading venues into NautilusTrader. They can be found in the top-level `adapters` subpackage.

An adapter typically comprises these components:



Component	Purpose
HttpClient	REST API communication.
WebSocketClient	Real-time streaming connection.
InstrumentProvider	Loads and parses instrument definitions from the venue.
DataClient	Handles market data subscriptions and requests.
ExecutionClient	Handles order submission, modification, and cancellation.

Instrument providers

Instrument providers parse venue API responses into Nautilus `Instrument` objects.

An `InstrumentProvider` serves two use cases:

- Standalone discovery of available instruments for research or backtesting
- Runtime loading in a `sandbox` or `live` [environment context](#) for actors and strategies

Research and backtesting

Here is an example of discovering the current instruments for the Binance Futures testnet:

```
import asyncio
import os

from nautilus_trader.adapters.binance.common.enums import BinanceAccountType
from nautilus_trader.adapters.binance.common.enums import BinanceEnvironment
from nautilus_trader.adapters.binance import get_cached_binance_http_client
from nautilus_trader.adapters.binance.futures.providers import BinanceFuturesInstrumentProvider
from nautilus_trader.common.component import LiveClock

async def main():
    clock = LiveClock()

    client = get_cached_binance_http_client(
        clock=clock,
        account_type=BinanceAccountType.USDT_FUTURES,
        api_key=os.getenv("BINANCE_FUTURES_TESTNET_API_KEY"),
        api_secret=os.getenv("BINANCE_FUTURES_TESTNET_API_SECRET"),
        environment=BinanceEnvironment.TESTNET,
    )

    provider = BinanceFuturesInstrumentProvider(
        client=client,
        account_type=BinanceAccountType.USDT_FUTURES,
    )

    await provider.load_all_async()

    # Access loaded instruments
    instruments = provider.list_all()
    print(f"Loaded {len(instruments)} instruments")
```

Live trading

Each integration handles this differently. An `InstrumentProvider` within a `TradingNode` generally offers two loading behaviors:

- Load all instruments on start:

```
from nautilus_trader.config import InstrumentProviderConfig

InstrumentProviderConfig(load_all=True)
```



- Load only the instruments specified in configuration:

```
InstrumentProviderConfig(load_ids=["BTCUSDT-PERP.BINANCE", "ETHUSDT-PERP.BINANCE"])
```



Data clients

Data clients handle market data subscriptions and requests for a venue. They connect to venue APIs and normalize incoming data into Nautilus types.

Requesting data

Actors and strategies can request data using built-in methods. Data returns via callbacks:

```
from nautilus_trader.model import Instrument, InstrumentId
from nautilus_trader.trading.strategy import Strategy

class MyStrategy(Strategy):
    def on_start(self) -> None:
        # Request an instrument definition
        self.request_instrument(InstrumentId.from_str("BTCUSDT-PERP.BINANCE"))

        # Request historical bars
        self.requestBars(BarType.from_str("BTCUSDT-PERP.BINANCE-1-HOUR-LAST-EXTERNAL"))

    def on_instrument(self, instrument: Instrument) -> None:
        self.log.info(f"Received instrument: {instrument.id}")

    def on_historical_data(self, data) -> None:
        self.log.info(f"Received historical data: {data}")
```



Subscribing to data

For real-time data, use subscription methods:

```
def on_start(self) -> None:
    # Subscribe to live trade updates
    self.subscribe_trade_ticks(InstrumentId.from_str("BTCUSDT-PERP.BINANCE"))
```



```
# Subscribe to live bars
self.subscribe_bars(BarType.from_str("BTCUSDT-PERP.BINANCE-1-MINUTE-LAST-EXTERNAL"))

def on_trade_tick(self, tick: TradeTick) -> None:
    self.log.info(f"Trade: {tick}")

def on_bar(self, bar: Bar) -> None:
    self.log.info(f"Bar: {bar}")
```



See the [Actors](#) documentation for a complete reference of available request and subscription methods with their corresponding callbacks.

Execution clients

Execution clients handle order management for a venue. They translate Nautilus order commands into venue-specific API calls and process execution reports back into Nautilus events.

Key responsibilities:

- Submit, modify, and cancel orders.
- Process fills and execution reports.
- Reconcile order state with the venue.
- Handle account and position updates.

The `ExecutionEngine` routes commands to the appropriate execution client based on the order's venue. See the [Execution](#) guide for details on order management from a strategy perspective.



For building a custom adapter, see the [Adapter Developer Guide](#).

Related guides

- [Live Trading](#) - Configure and run live trading with adapters.
- [Execution](#) - Order execution through adapters.
- [Data](#) - Market data provided by adapters.

[< Live Trading](#)

[Previous Page](#)

[Rust >](#)

[Next Page](#)